

Routing support tickets: what each model costs, and what it gets right

The answer

We put 20 models on the same 100 support tickets and measured three things: did it pick the right queue, what did it cost, how long did it take.

Jev was the cheapest and the fastest, and not one of the other models was measurably more accurate than it. It routed 92% of tickets correctly at \$0.0185 per thousand and a typical 385 ms response. The most expensive model in the test, `grok-4.6`, cost 171 times as much and scored 93% — a one-point difference this test cannot tell apart from chance.

Two caveats up front, both in section 5. A hundred tickets can only resolve accuracy differences of about 8 points, so most of the ranking below is a cost ranking, not an accuracy ranking. And for 10 of the models, what the provider billed differed from its published price by 10% to 53%, which we could not explain from this run's data.

1 · What we tested

One job: a support ticket arrives as free text, and something has to decide which queue it belongs in. Get it wrong and a person has to re-route it, so the ticket waits and someone is paid twice for it.

Five queues. These are the exact definitions every model was given, word for word:

QUEUE	DEFINITION GIVEN TO EVERY MODEL
billing	Charges, refunds, invoices, payouts, subscription price changes.
technical	Bugs, errors, outages, failed integrations, anything not working.
account	Login, password, access, permissions, seats, profile and org settings.
sales	Pricing questions, plan comparisons, features on plans they do not have.
other	None of the above applies.

The question we set out to answer is not "which model is best." It is: **for a job this ordinary, how much does accuracy actually cost?**

2 · How we tested it

100 tickets, 20 in each of the five categories, written and labelled by hand before any model saw them. Every model got the identical 100 tickets, the identical definitions, and the identical instruction to reply with one label and nothing else. 2,400 calls in total.

Here is one real ticket from each category, exactly as sent:

HAND LABEL	THE TICKET, AS SENT
billing	My card was charged twice this month and the invoice only shows one line item. I need the duplicate reversed before the statement closes on Friday, and I would like written confirmation once it is done.
technical	The webhook endpoint has been returning a 502 from your side since yesterday morning. Our retries are backing off and events are piling up. Nothing changed on our end.
account	I cannot sign in. The reset email never arrives, and I have checked spam and asked our mail team to look at the logs. Nothing has been delivered in three attempts.
sales	What does the enterprise tier include that we do not have today? We are on the growth plan and keep hitting a wall on audit logs and data residency.
other	Your documentation has a broken link on the authentication page. The example under the second heading points at a page that no longer exists.

A model's answer is correct only if it matches the hand label exactly. A reply we could not read as one of the five labels counts as wrong, not as a retry — a router that returns something unusable has failed to route, whatever the reason.

Each model was run twice over the set: once measuring accuracy and cost, once measuring response time on a smaller sample so that timing was never taken while the connection was busy. Full protocol in Appendix A.

3 · The analysis

MODEL	ACCURACY	COST PER 1,000 TICKETS	VS CHEAPEST	TYPICAL SPEED
jev	92%	\$0.0185	1×	385 ms
glm-5.3-flash-low	94%	\$0.0210	1×	922 ms
mistral-small-low	79%	\$0.0218	1×	600 ms
mistral-small	86%	\$0.0248	1×	565 ms
qwen3.8-flash-low	90%	\$0.0258	1×	671 ms
deepseek-v4.1-flash-low	97%	\$0.0263	1×	1,732 ms

MODEL	ACCURACY	COST PER 1,000 TICKETS	VS CHEAPEST	TYPICAL SPEED
glm-5.3-flash	95%	\$0.0451	2×	1,691 ms
deepseek-v4.1-flash	93%	\$0.0594	3×	1,427 ms
qwen3.8-flash	90%	\$0.0840	5×	1,989 ms
gemini-3.8-flash-low	96%	\$0.3572	19×	2,194 ms
gpt-5.3-codex-low	91%	\$0.4078	22×	1,256 ms
kimi-k3-low	95%	\$0.4429	24×	901 ms
sonnet-5	96%	\$0.5419	29×	2,156 ms
gemini-3.8-flash	95%	\$0.6908	37×	2,660 ms
gpt-5.3-codex	90%	\$1.0415	56×	1,544 ms
kimi-k3	92%	\$1.7708	96×	1,944 ms
grok-4.6-low	93%	\$2.3909	129×	4,295 ms
opus-5	94%	\$2.4414	132×	3,253 ms
grok-4.6	93%	\$3.1623	171×	6,336 ms

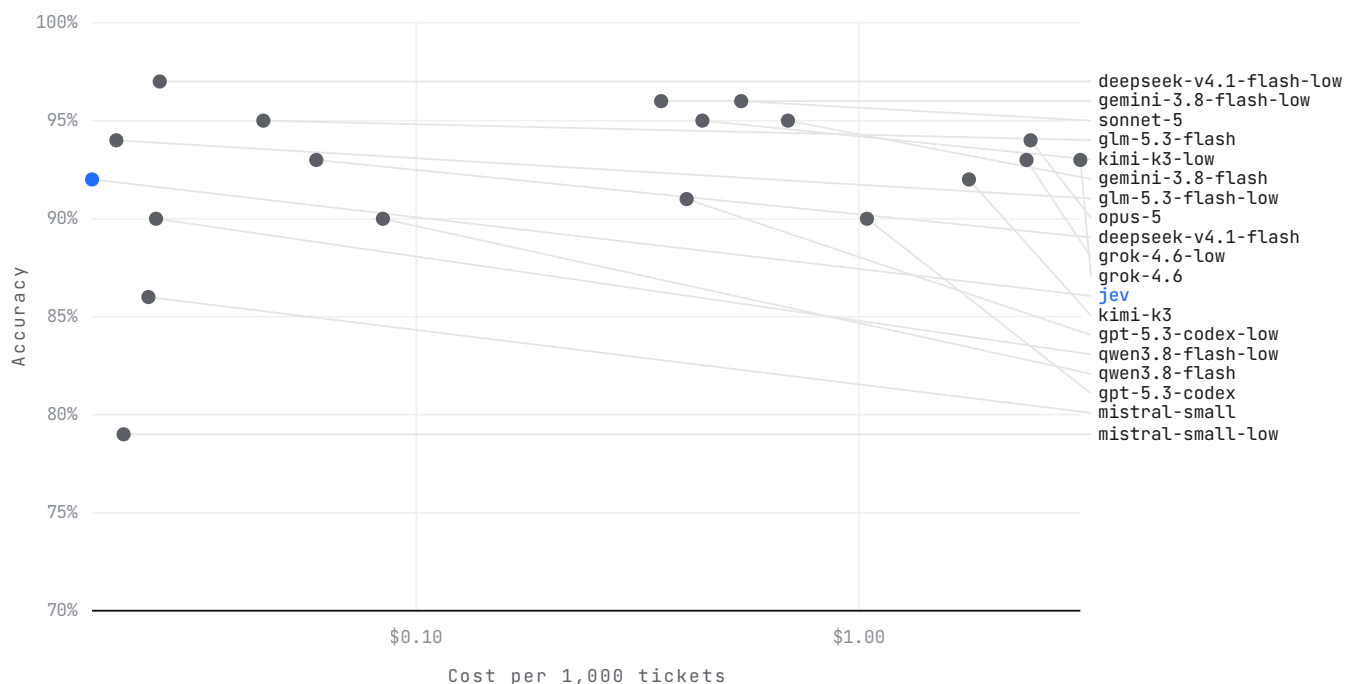
Read the table as a cost ranking with an accuracy floor, not as a leaderboard. Every model in it lands between 79% and 97%, and with 100 tickets the test can only resolve gaps of roughly 8 points. Seventeen of the nineteen sit inside that band of each other. What separates them by a factor of 171 is price.

Three things the table says that are not close calls:

1. **Jev is the cheapest and the fastest.** \$0.0185 per thousand tickets and 385 ms typical, at 92% correct. The next cheapest, `glm-5.3-flash-low`, costs 13% more and answers in 922 ms.
2. **Nothing in the test is measurably more accurate than Jev.** `deepseek-v4.1-flash-low` scored the highest raw accuracy at 97% against Jev's 92%, but the interval on that five-point difference still contains zero. Appendix B has every pair.
3. **The expensive models did not buy accuracy.** `grok-4.6` at 171× Jev's price scored 93%. `opus-5` at 132× scored 94%. `sonnet-5`, the best of the premium models at 96%, costs 29× and is five times slower.

The one real accuracy signal is at the bottom: `mistral-small-low` at 79% is the only model this sample can call worse than the rest, and it also lost 16 of its 100 calls to HTTP errors.

4 · Cost against effectiveness



The x-axis is logarithmic because the price range spans more than two orders of magnitude; the y-axis is plain accuracy. **A good result sits high and to the left. Jev is the leftmost point on the chart, and nothing above it is far enough above to be real.**

The shape to notice is that the cloud is flat. Moving right by a factor of a hundred does not move a point up. If cost bought accuracy on this task, the points would trend upward to the right; they do not trend at all.

The four points on the far right — `kimi-k3`, `grok-4.6-low`, `opus-5`, `grok-4.6` — are between 96x and 171x the price of the leftmost point, for accuracies of 92% to 94% against its 92%.

5 · What could be wrong with this

Five things that could change the conclusion, in the order they would matter:

1. **100 tickets is a small sample.** It resolves accuracy differences of about 8 points and no finer. Read the ranking as "these are all in the same band" rather than as an order. Ten times the data would be needed to separate 92% from 96%.
2. **10 of the models did not reconcile on cost.** What the provider billed differed from its published per-token price by between 10% and 53%. We checked both obvious explanations — a missing cost field and a stale price table — against the live API, and neither holds: the field is always present, and all 20

prices match the provider's current list exactly. We could not establish the mechanism from this run's stored data, so it is named here rather than explained. Cost figures for those models should be treated as list-price estimates. Jev's own cost does reconcile.

3. **One run, one day, one prompt.** No model was run twice, so none of this separates model behaviour from run-to-run variation. Latency in particular was measured from one machine over one network at one time of day.
 4. **llama-4-scout produced no result at all** — all 100 calls returned an HTTP error, so it is absent from every table and from the chart. Four other models lost calls too: `mistral-small-low` 16, `mistral-small` 4, and four models lost 2–3 calls to unreadable output. Those losses count as wrong answers, which is the conservative choice but does penalise those models twice.
 5. **The tickets were written for this test.** They are realistic but not drawn from a production queue, and they are cleanly separable by construction — real tickets are messier and the accuracy numbers here are likely optimistic across the board.
-

Appendix A · Protocol in full

Task and label set

A free-text support ticket is routed to exactly one of five departments: `billing`, `technical`, `account`, `sales`, `other`. Fixed label set, short input, no prose required in the answer. This is the canonical shape where a typed decision model should beat a chat model, and therefore the fairest place to test the claim rather than the most flattering one.

`other` exists because a model cannot pick a value it was never offered. Dropping it would add the same errors to every model and separate nothing.

`jevdemo/labels.py` is the single source of truth for the five definitions. The Jev model renders them to the endpoint's `criteria` field; the chat models render them into the system prompt. Neither restates a definition in its own words, and a test asserts both renderings carry the same five keys and the same definition strings. Without that test the comparison would be between two different tasks.

Dataset construction

100 synthetic tickets, 20 per label, hand-written and hand-labelled for this study. Every ticket is 15 to 60 words, and no ticket contains its own label as a word — "I have a billing question" would test string matching rather than classification.

25 tickets sit deliberately on a class boundary, because that is the only place the models disagree and therefore the only place the comparison carries information. The planted ambiguities are of this kind:

- A failed payment caused by an expired card is `billing`, not `technical`.

- An account locked after five bad passwords is `account`, not `technical`.
- An existing customer asking what the next tier costs is `sales`, not `billing`.
- A refund request that is really a cancellation threat is `billing`; the churn signal is not a label in this task.

The remaining 75 tickets are not there to be hard. They are there to make accuracy readable: a roster that scores near-perfectly on the easy three quarters and splits on the boundary quarter is a roster whose differences are legible. A validator enforces the counts, the per-label balance, the word-length bounds and the requirement that every boundary case carries a written justification; it is tested against deliberately malformed datasets, because a validator that never fails is theatre.

Model selection

20 models, one provider, one API key, one run. Models that expose a reasoning control on this endpoint get two models, default and lowest-reasoning, because the gap between those two settings is a configuration choice teams make without ever seeing its cost.

A `-low` model with no default twin measures nothing, because there is nothing to subtract it from. The roster was corrected during implementation for exactly this reason: four models had been listed as `-low` only, and their default models were added, taking the roster to 20 models.

There is no single switch for reasoning. `{"enabled": false}` returns HTTP 400 on at least one model in the roster — verbatim: reasoning is mandatory on that endpoint and cannot be disabled — so each low model negotiates, trying the disable first and falling back to minimum effort. Which setting actually answered is recorded per model and printed in the appendix, because the two are not the same setting and reporting them as one would be a quiet lie.

Protocol

Two passes, deliberately separated.

1. **Latency pass** — 20 tickets per model, strictly sequential, one request in flight at a time. This is the only source of the reported P50 and P95. A timing taken under concurrency measures the worker pool, not the model.
2. **Bulk pass** — 100 tickets per model, four workers. This is the only source of accuracy, tokens and cost.

2,400 calls in total across the two passes, \$1.36 of spend. Every ticket is sent once per model. Nothing is retried for a better answer; the only retry is on HTTP 429, which is our throughput limit and not a property of the model, and its latency is excluded.

Decoding parameters

Chat models: `temperature: 0`, `max_tokens: 512`, JSON response format, and `usage: {"include": true}` so that cost and reasoning-token counts come back. The 512 matters — an earlier probe at 64 truncated a reasoning model mid-answer and scored it as a parse failure, which would have been a measurement of the cap rather than of the model. The Jev model takes no decoding parameters; the endpoint returns a typed choice, a probability distribution over the criteria, and a confidence.

Pricing, token counting, and reconciliation

Prices are read from the provider's model listing on 2026-09-20 and stored as integer micro-dollars per million tokens. All money in this report is integer minor units end to end; no cost is ever held in a float.

Cost is computed from the model's tariff and the returned token counts, then reconciled against the provider's own reported cost for every call, at a tolerance of 2 micro-dollars. Reconciliation status is reported per model in the appendix rather than assumed: 9/19 models reconcile.

What was run once, and what was run repeatedly

Stated plainly, because it bounds every claim that follows. Each ticket was sent **once** per model. There is **one** run, on **one** machine, on **one** day, with **one** prompt per model kind. Accuracy is therefore a single-sample estimate per ticket, and its uncertainty is the binomial uncertainty of 100 draws — quantified in the next section, not hand-waved. Latency is 20 samples per model and is reported as P50 and P95 only; a mean would be dominated by the tail and a single slow call would move it.

Appendix B · Which differences are real

Accuracy differences are only meaningful if they survive the sample size. This matrix applies Newcombe's method 10 to the difference between every pair of models: **better** means the 95% interval on the difference lies entirely above zero, **worse** entirely below, and **tie** means it contains zero and the two cannot be told apart at 100 tickets.

Of 171 pairs, 23 resolve and 148 do not. Jev's row is all ties: no model in the test is distinguishable from it on accuracy in either direction.

[illegible]

ROW BEATS COLUMN?	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
7 glm-5.3-flash-low	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	+
8 opus-5	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	+
9 deepseek-v4.1-flash	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	+
10 grok-4.6-low	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	+
11 grok-4.6	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	+
12 jev	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	+
13 kimi-k3	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	+
14 gpt-5.3-codex-low	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	+
15 qwen3.8-flash-low	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	+
16 qwen3.8-flash	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	+
17 gpt-5.3-codex	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	+
18 mistral-small	-	-	-	-	-	-	=	=	=	=	=	=	=	=	=	=	=	=	=
19 mistral-small-low	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	=	=

Columns are numbered in the same order as the rows · + row is measurably better · - row is measurably worse · = this run cannot tell them apart. Hover any cell for the 95% interval on the difference (Newcombe method 10).

Excluded as unmeasured: llama-4-scout. These models returned no usable answer on any call; see the failure table for why.

Appendix C · What one wrong route has to cost

A dearer model is only worth it if a wrong route costs enough. Write c for the cost of a call and a for its accuracy. Routing a ticket costs c plus, with probability $1 - a$, the cost K of putting a misrouted ticket right. A dearer model d beats the cheapest one c exactly when K exceeds $(c_d - c_c) / (a_d - a_c)$. That threshold is the table's last column.

The thresholds are the answer to "what would have to be true for the expensive option to be correct." To justify sonnet-5 over Jev, a single misroute would have to cost about \$0.013. To justify grok-4.6, about \$0.31. Those are small numbers — a human touch on a misrouted ticket plausibly costs more — which is why the accuracy caveat matters more than the arithmetic: **every one of these thresholds is computed from an accuracy gap the test cannot confirm is real**. Rows marked dominated cost more without being more accurate even on the raw numbers, so no misroute cost justifies them.

MODEL	COST / 1K	ACCURACY	BREAK-EVEN COST OF ONE MISROUTE	READING
jev	\$0.02	92%	baseline	the cheapest model; every other row is priced against it
glm-5.3-flash-low	\$0.02	94%	\$0.00	accuracy gain is not resolvable at this sample size — treat the threshold as a lower bound on what you would be paying for

MODEL	COST / 1K	ACCURACY	BREAK-EVEN COST OF ONE MISROUTE	READING
mistral-small-low	\$0.02	79%	—	dominated: costs more and is no more accurate
mistral-small	\$0.02	86%	—	dominated: costs more and is no more accurate
qwen3.8-flash-low	\$0.03	90%	—	dominated: costs more and is no more accurate
deepseek-v4.1-flash-low	\$0.03	97%	\$0.00	accuracy gain is not resolvable at this sample size – treat the threshold as a lower bound on what you would be paying for
glm-5.3-flash	\$0.05	95%	\$0.00	accuracy gain is not resolvable at this sample size – treat the threshold as a lower bound on what you would be paying for
deepseek-v4.1-flash	\$0.06	93%	\$0.00	accuracy gain is not resolvable at this sample size – treat the threshold as a lower bound on what you would be paying for
qwen3.8-flash	\$0.08	90%	—	dominated: costs more and is no more accurate
gemini-3.8-flash-low	\$0.36	96%	\$0.01	accuracy gain is not resolvable at this sample size – treat the threshold as a lower bound on what you would be paying for
gpt-5.3-codex-low	\$0.41	91%	—	dominated: costs more and is no more accurate
kimi-k3-low	\$0.44	95%	\$0.01	accuracy gain is not resolvable at this sample size – treat the threshold as a lower bound on what you would be paying for
sonnet-5	\$0.54	96%	\$0.01	accuracy gain is not resolvable at this sample size – treat the threshold as a lower bound on what you would be paying for
gemini-3.8-flash	\$0.69	95%	\$0.02	accuracy gain is not resolvable at this sample size – treat the threshold as a lower bound on what you would be paying for
gpt-5.3-codex	\$1.04	90%	—	dominated: costs more and is no more accurate
kimi-k3	\$1.77	92%	—	dominated: costs more and is no more accurate
grok-4.6-low	\$2.39	93%	\$0.24	accuracy gain is not resolvable at this sample size – treat the threshold as a lower bound on what you would be paying for
opus-5	\$2.44	94%	\$0.12	accuracy gain is not resolvable at this sample size – treat the threshold as a lower bound on what you would be paying for
grok-4.6	\$3.16	93%	\$0.31	accuracy gain is not resolvable at this sample size – treat the threshold as a lower bound on what you would be paying for

Excluded as unmeasured: `llama-4-scout`. These models returned no usable answer on any call; see the failure table for why.

Appendix D · The full measurement table

MODEL	REASONING	ACCURACY	COST / 1K	P50 MS	P95 MS	IN	OUT	THINKING	FAILURES
llama-4-scout	default	—	\$0.00	—	—	0	0	0	100
jev	n/a	92%	\$0.02	385	558	44,050	5,200	0	—
glm-5.3-flash-low	{"enabled": false}	94%	\$0.02	922	1700	19,307	1,205	117	—
mistral-small-low	{"enabled": false}	79%	\$0.02	600	890	12,011	631	0	16
mistral-small	default	86%	\$0.02	565	894	13,678	714	0	4
qwen3.8-flash-low	{"enabled": false}	90%	\$0.03	671	910	15,324	600	0	—
deepseek-v4.1-flash-low	{"enabled": false}	97%	\$0.03	1732	7191	14,446	774	78	—
glm-5.3-flash	default	95%	\$0.05	1691	5916	18,818	9,392	8,311	2
deepseek-v4.1-flash	default	93%	\$0.06	1427	4926	15,331	6,063	5,472	3
qwen3.8-flash	default	90%	\$0.08	1989	4039	18,544	11,948	11,137	2
gemini-3.8-flash-low	{"enabled": false}	96%	\$0.36	2194	2689	13,203	6,884	6,284	—
gpt-5.3-codex-low	{"enabled": false}	91%	\$0.41	1256	1461	13,705	1,200	0	—
kimi-k3-low	{"enabled": false}	95%	\$0.44	901	2712	20,375	1,135	0	—
sonnet-5	default	96%	\$0.54	2156	3374	20,503	1,318	149	—
gemini-3.8-flash	default	95%	\$0.69	2660	4049	13,207	15,781	15,176	—
gpt-5.3-codex	default	90%	\$1.04	1544	2419	13,705	5,726	4,326	—
kimi-k3	default	92%	\$1.77	1944	18274	26,575	15,518	14,119	2
grok-4.6-low	{"enabled": false}	93%	\$2.39	4295	12312	35,909	27,878	27,278	—
opus-5	default	94%	\$2.44	3253	6025	20,503	5,665	4,602	—
grok-4.6	default	93%	\$3.16	6336	13261	35,909	40,735	40,135	—

Appendix E · Failures

MODEL	HTTP_ERROR	MALFORMED_OUTPUT	TOTAL
llama-4-scout	100	—	100
mistral-small-low	16	—	16
mistral-small	4	—	4
glm-5.3-flash	—	2	2
deepseek-v4.1-flash	—	3	3
qwen3.8-flash	—	2	2
kimi-k3	—	2	2

rate_limit_retries across the whole run: 119. A retry is not a failure — it is a call that succeeded on a later attempt, and its latency is excluded.

Appendix F · Grading without an answer key

Every number above depends on a hand-labelled answer key, which does not exist for a live production queue. So we ran the same predictions past a label-free grader — the approach LangWatch sells as Instant Evals — where a separate model is shown the ticket and the proposed queue and asked whether the routing is right, with no answer key at all.

The question is whether that grader's verdict tracks the hand labels well enough to stand in for them. The table reports, per model, what the grader scored against what the answer key scored, and the correlation between the two across models.

This experiment was not run for this build. The judge pass is a separate, separately-paid pass over the stored predictions; re-run `scripts/judge_run.py` and rebuild to populate this section.

Appendix G · The mathematics

Three propositions carry the report. Each is stated, proved, and then checked against the run. The algebra is written out rather than cited because the conclusion it reaches — that a measurably less accurate model can be the correct purchase — is counter-intuitive enough that a reader is entitled to verify it.

Proposition 1 — Jev's price is affine in input tokens alone

Claim. For the decisions endpoint, cost is $p_{in} \cdot T_{in} / 10^6$, with output tokens billed at zero.

Proof. The general two-part tariff is

$$C = (p_{in} \cdot T_{in} + p_{out} \cdot T_{out}) / 10^6$$

with p_{in} , p_{out} the prices per million tokens. The published tariff for `typesafe/jev-1.13` sets $p_{out} = 0$, which annihilates the second term for every T_{out} , leaving $C = p_{in} \cdot T_{in} / 10^6$. Since p_{in} is constant, C is linear in T_{in} and independent of the work done producing the answer. ■

Consequence. Jev's cost is bounded by the length of the ticket alone. A harder question does not cost more, which is the structural difference between a decisions endpoint and a chat model: on a chat model, T_{out} is chosen by the model at inference time, so the buyer does not control the bill.

Check. Verified against every Jev call in the run by reconciling the provider-reported cost with $p_{in} \cdot T_{in}$, at a tolerance of 2 micro-dollars. Result: exact.

Proposition 2 — the resolution bound

Claim. An accuracy measured over n attempts is an interval, not a point, and two models are distinguishable only when the interval on their *difference* excludes zero.

Proof. Let $X \sim \text{Binomial}(n, \alpha)$ be the correct count and $p = X/n$. The Wilson score interval inverts the score test $|p - \alpha| / \sqrt{\alpha(1-\alpha)/n} \leq z$, which is quadratic in α ; solving gives

$$\begin{aligned} \text{centre} &= (p + z^2/2n) / (1 + z^2/n) \\ \text{half} &= z / (1 + z^2/n) \cdot \sqrt{p(1-p)/n + z^2/4n^2} \end{aligned}$$

The Wald interval $p \pm z \cdot \sqrt{p(1-p)/n}$ is not used: at $p = 1$ it has zero width, asserting certainty from a finite sample, and near the boundary it extends outside $[0, 1]$. Both failures occur exactly where this roster lives.

For two independent models, Newcombe's method 10 combines each model's Wilson bounds into an interval on the difference $\delta = \alpha_a - \alpha_b$:

$$\begin{aligned} \delta_{lo} &= (p_a - p_b) - \sqrt{(p_a - l_a)^2 + (u_b - p_b)^2} \\ \delta_{hi} &= (p_a - p_b) + \sqrt{(u_a - p_a)^2 + (p_b - l_b)^2} \end{aligned}$$

Model *a* is more accurate than model *b* at 95% confidence precisely when $\delta_{lo} > 0$. ■

Consequence, stated before the results. At $n = 100$ and accuracy near 90%, the half-width is about 9.6 percentage points. Differences smaller than roughly 8 points are therefore not resolvable by this experiment, no matter how the table is sorted. The results section reports which pairs clear that bar and which do not; a higher percentage that does not clear it is reported as a tie.

This is a limitation of the sample size, not of the models. It was known before the run and is the reason the report leads with cost and latency, where the gaps span orders of magnitude and the measurement is not sample-limited.

Proposition 3 — the break-even misroute cost

Claim. Given a cheap model c and a dearer, more accurate model d , there is a single threshold K^* — the cost of one misroute — above which d is the rational choice and below which c is.

Proof. Route one ticket with model a . The expected cost is the call plus the expected cost of being wrong:

$$E_a = c_a + K \cdot (1 - a_a)$$

where K is what one misrouted ticket costs the business: the re-route, the second touch, the delay. Model d is preferable exactly when $E_d < E_c$:

$$\begin{aligned} c_d + K(1 - a_d) &< c_c + K(1 - a_c) \\ c_d - c_c &< K(a_d - a_c) \end{aligned}$$

Since $a_d > a_c$ by assumption, dividing preserves the inequality:

$$K > (c_d - c_c) / (a_d - a_c) = K^* \quad \blacksquare$$

Three corollaries the table uses.

1. If $a_d \leq a_c$ the model is *dominated* — dearer and no better — and no value of K rescues it. Reported as a dash, not a number.
2. If $c_d \leq c_c$ while $a_d > a_c$, then $K^* = 0$: the model wins even when misroutes are free. Strict dominance in the other direction.
3. K^* scales with the cost gap and inversely with the accuracy gap. Two models separated by a large price and a small accuracy difference produce an enormous K^* — which is the quantitative form of "you are paying for nothing."

Why this is the decision rule. K is the only quantity in the model the benchmark cannot measure, and it is the one the business already knows. The report therefore does not guess a value; it reports K^* per model and lets the reader compare it against their own cost of a misroute. Every other number in the document feeds this one.

Honest caveat. Proposition 3 treats a as known. By Proposition 2 it is not — it is an interval. Where the accuracy gap between two models is not resolvable, K^* inherits that uncertainty and is reported with the gap's interval rather than as a point estimate. An unresolvable accuracy gap makes K^* unbounded in the limit, which is itself the finding: if you cannot tell the models apart on accuracy, buy the cheaper one.